KUBERNETES COMPREHENSIVE · AXP-K8S-5D

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	14 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 5 — Security, Packaging and Running It

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 5

What changed today

Yesterday	Today
Every pod carried an API token it never used	One workload has a token, and the reason is written down
Anyone with cluster access could do anything	Four roles, seven bindings, every grant provable
107 YAML files applied by hand	One chart, five configurations, one command
No idea how staging differs from production	The difference is a diff
The SLO was an opinion	The SLO is a query
Every incident reported by a merchant	Nine alerts on symptoms a merchant would feel

5.1 ServiceAccounts and Pod Security Admission

1. What it is

A **ServiceAccount** is an identity for a pod. Every namespace has one called `default`, and unless you say otherwise every pod uses it and receives a signed JWT mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token`.

Pod Security Admission is a built-in admission controller that evaluates every pod against one of three standards — `privileged`, `baseline`, `restricted` — and either rejects it, records it, or warns about it.

They answer two different questions and are frequently confused:

	Question	Enforced by
ServiceAccount	*Who is this pod?*	Authentication
Pod Security Admission	*Is this pod allowed to exist in this shape?*	Admission control

A pod can have a perfect ServiceAccount and still run as root with a hostPath mount.

2. Why it exists

Kubernetes needs a way for workloads to authenticate to the API server, because some of them legitimately need to call it — an ingress controller watching Ingress objects, a metrics agent listing Nodes, an operator reconciling its own CRs. The ServiceAccount is that mechanism.

The mounting-by-default behaviour is historical. It made the common case easy in 2015, when the common case was in-cluster tooling. For application workloads it is now a liability with no compensating benefit.

Pod Security Admission exists because PodSecurityPolicy — its predecessor — was removed in v1.25. PSP was powerful and almost impossible to reason about: which policy applied to a pod depended on a ranking of the policies the *requesting user* was authorised to use. PSA replaced it with something deliberately simpler: three named standards, applied per namespace by a label.

3. The business problem

AxisPay's penetration test produced one sentence: *"A remote-code-execution vulnerability in any AxisPay service yields a Kubernetes API token with the permissions of the `default` ServiceAccount in that namespace."*

The platform team's first reaction was that `default` has no permissions. The tester's reply is the correct one: that is a statement about today's RBAC configuration, not a control. The token is a valid credential that identifies the namespace, reveals the API server's address, and will keep working. Change RBAC next quarter — grant something to `default` because it is convenient — and the finding becomes an incident retroactively.

4. How it works

The token. When a pod is admitted, if `automountServiceAccountToken` is not `false` on either the ServiceAccount or the pod spec, the kubelet projects a token into the container. Since v1.22 these are **bound service account tokens**: time-limited, audience-scoped, and tied to the pod's lifetime. That is a real improvement — a stolen token expires — but it is still a live credential while the pod runs.

Turning it off. Two places, and the pod spec wins:

```
# on the ServiceAccount — applies to every pod using it
automountServiceAccountToken: false
---
# on the pod spec — overrides the ServiceAccount
spec:
  automountServiceAccountToken: false
```

The three standards, cumulatively:

Standard	Blocks
`privileged`	Nothing.
`baseline`	Privileged containers, host namespaces, **hostPath volumes**, host ports, added capabilities beyond the default set, unsafe sysctls, custom SELinux/AppArmor/proc-mount settings.
`restricted`	Everything in baseline, plus: must run as non-root, must drop ALL capabilities, must set `allowPrivilegeEscalation: false`, must set a seccomp profile, and volume types are limited.

The label form is `pod-security.kubernetes.io/<MODE>: <LEVEL>`, plus an optional `-version` suffix:

```
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

5. Internal architecture

PSA is a **validating admission controller compiled into the API server**. There is no webhook, no extra pod, and therefore no dependency that can be down when you need admission to work. Its evaluation happens after authentication and authorisation and before persistence, which is why a rejected pod never exists at all.

Bound tokens are issued by the `TokenRequest` API and projected by the kubelet as a **projected volume**, refreshed before expiry. The kubelet, not the pod, does the refreshing.

6. Component interactions

```
kubectl apply
   → API server: authenticate (who are you?)
   → API server: authorise via RBAC (may you create a pod here?)
   → API server: admission — mutating, then VALIDATING (PSA runs here)
   → etcd: persist
   → scheduler: assign a node
   → kubelet: project the token, start the container
```

Two consequences follow from PSA's position in that chain:

1. **Rejection happens before persistence.** No object, no CrashLoopBackOff, no cleanup, no window in which it ran.
2. **PSA evaluates on create and update, not continuously.** Existing pods keep running when you tighten a namespace label. The failure arrives at the next rollout, which may be weeks later and will not look connected to the label change.

7. Enterprise example

A retail bank tightened `enforce` from `baseline` to `restricted` across forty namespaces in a single change, on a Friday. Nothing broke. Three weeks later a routine release of a payments service failed with a rejection nobody could explain, and the on-call engineer — reasonably — reverted the release rather than the namespace label. It took two more failed releases before anyone connected the two changes.

The correct sequence was available and free: set `warn` and `audit` to `restricted` first, leave `enforce` at `baseline`, and let three weeks of warnings size the work.

8. Real-world analogy

A **ServiceAccount** is a staff badge. A **token** is that badge being in your pocket right now. The bank's rule is that you carry your badge only in the areas where you need it — because a badge in a stolen coat is a badge someone else has.

Pod Security Admission is the metal detector at the door. It does not care who you are; it cares what you are carrying. And it is at the door, not roaming the building — which is why it stops you coming in and does not eject you if the rules change while you are inside.

9. Best practices

- **`automountServiceAccountToken: false` as the default.** Turn it on for the one workload that needs it and write down why, next to the setting.
- **One ServiceAccount per service** that has any API access at all. Shared identities make an audit log unreadable.
- **`enforce` what you meet today; `warn` and `audit` where you intend to be.** The gap is a to-do list.
- **`restricted` on every namespace holding application code.** Exceptions belong to infrastructure that genuinely needs host access, and each one gets a comment.

- **Set `enforce-version`** rather than leaving it implicit if you need reproducibility across cluster upgrades — the standards do evolve.

10. Common mistakes

Mistake	What happens
Assuming `default` having no permissions makes the token safe	It is still a valid credential. It authenticates. RBAC is the only thing stopping it, and RBAC changes.
Turning on `restricted` everywhere at once	Nothing breaks today. Rollouts fail weeks later, apparently unconnected.
Setting `baseline` where a hostPath is needed	**Baseline forbids hostPath.** Log collectors and node agents cannot run under it. This is the mistake that silently kills a log pipeline.
Using one ServiceAccount for every workload	The audit log tells you a pod did something. It cannot tell you which one.
Believing PSA re-evaluates running pods	It does not. Tightening a label is not a remediation.

11. Security considerations

A mounted token is a **credential at rest inside a process you do not fully control**. The threat model is not "someone stole a YAML file"; it is "a dependency in your Python image had a deserialisation bug".

Bound tokens reduce the blast radius — audience-scoped, time-limited, pod-bound — but the reduction is in duration, not in kind.

Note also what PSA does **not** do: it does not restrict what an image contains, whether it is signed, or where it came from. Those are separate controls (image policy webhooks, signature verification) and this course does not cover them.

12. Performance considerations

PSA costs a few microseconds per admission and is in-process. There is no measurable overhead and no reason to disable it for performance.

Token projection costs a file descriptor and a periodic refresh. Turning it off for twelve services saves nothing measurable — the argument for turning it off is entirely about risk.

13. High availability

PSA is compiled into the API server, so it has exactly the availability of the API server itself. Compare with a validating webhook, which introduces a new failure mode: if the webhook is down and its `failurePolicy` is `Fail`, no pods can be created anywhere. Several outages have been caused by a policy webhook that was itself unschedulable.

14. Disaster recovery

Namespace labels are part of the namespace object, so `kubectl get ns -o yaml` captures them. Keep them in the repository — `manifests/day5/security/01-pod-security.yaml` — and they are restored with everything else.

ServiceAccounts restore trivially. Their tokens do not need to: bound tokens are minted on demand, so there is nothing to back up.

15. Monitoring

```
# admission rejections, by reason
sum by (reason) (rate(apiserver_admission_webhook_rejection_count[5m]))
```

More usefully in practice, PSA `audit` mode writes to the API server audit log with `pod-security.kubernetes.io/audit-violations` annotations. Ship those and you have a live list of everything that would break if you tightened `enforce`.

16. Troubleshooting

Symptom	Cause	Command
`forbidden: violates PodSecurity`	The pod does not meet the namespace's standard	`kubectl get ns <ns> --show-labels`
Pod created with a warning	`warn` is stricter than `enforce`	Expected — it is the migration signal
A token is still mounted after the change	Old pods	`kubectl rollout status` — new pods only
Log collector will not start	`baseline` forbids hostPath	The namespace needs `privileged`
Existing pods unaffected by a label change	PSA evaluates on create/update	Roll the workload to find out

Interview questions

1. **A pod uses the `default` ServiceAccount and RBAC grants it nothing. Is the mounted token a security problem?** — Yes. It is a valid credential that authenticates; only current RBAC stops it, and RBAC changes. It also reveals the namespace and the API server address.
2. **What is the difference between a 401 and a 403 from the API server using a pod's token?** — 401 means no valid credential. 403 means the credential was accepted and then denied by authorisation.
3. **You set `enforce: restricted` on a namespace with running non-compliant pods. What happens?** — Nothing immediately. They keep running. The next create or update is rejected.

4. **Why can a log collector not run under `baseline`?** — Baseline forbids hostPath volumes, and it needs one to read `/var/log/pods`.
5. **What does `warn` give you that `audit` does not?** — It reaches the human applying the change, at the moment they can act.
6. **Name two ways to read a Secret without any `secrets` RBAC grant.** — `pods/exec` into a pod that consumes it; `create` a pod that mounts it. (Also `pods/portforward` to a service that exposes it.)

5.2 RBAC

1. What it is

Role-Based Access Control. Four object kinds:

- **Role** — a set of permissions, valid in one namespace.
- **ClusterRole** — a set of permissions, not scoped to a namespace.
- **RoleBinding** — grants a Role *or a ClusterRole* to subjects, within one namespace.
- **ClusterRoleBinding** — grants a ClusterRole to subjects, cluster-wide.

Subjects are Users, Groups or ServiceAccounts. Kubernetes has no User objects — users come from your authentication layer (certificates, OIDC, a cloud IAM integration) and RBAC only ever sees the resulting name.

2. Why it exists

Every request to the API server is authorised, and RBAC is the mechanism almost everyone uses. It is deliberately simple: a request matches a rule or it does not. There is no priority, no ordering, no conditional logic, and — critically — **no deny**.

That simplicity is the design. A policy language with denies and precedence is one where "can Alice read this Secret?" requires evaluating the whole policy set. In RBAC the answer is: does any rule grant it? If not, no.

3. The business problem

An external auditor needs read access to the AxisPay namespaces for six weeks. Two requirements from compliance:

1. They must be able to read **everything** — workloads, config, events, logs — because "we could not see it" invalidates the audit.
2. They must **never** read a Secret. Those hold the database password and the JWT signing key, and an auditor with the signing key can mint any merchant's session.

These look contradictory. They are not, and the resolution is the most useful thing to understand about RBAC: you do not deny Secrets. You write a role that never mentions them.

4. How it works

A rule is a cross-product of API groups, resources and verbs:

```
rules:
  - apiGroups: [""]                 # "" is the core group
    resources: ["pods", "services", "configmaps", "events"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]          # a SUBRESOURCE — a separate grant
    verbs: ["get", "list"]
  - apiGroups: ["apps"]
    resources: ["deployments", "statefulsets"]
    verbs: ["get", "list", "watch"]
```

Four rules decide everything:

1. **Purely additive.** No deny exists. A permission is absent, never revoked.
2. **The union of all bindings wins.** Two bindings granting different things grant both.
3. **ClusterRole + RoleBinding is the useful combination.** The ClusterRole defines *what*; the RoleBinding decides *where*. One permission set, bound in three namespaces.
4. **Subresources are separate resources.** `pods`, `pods/log`, `pods/exec`, `pods/portforward`, `pods/eviction` are five independent grants.

The 2×2, including the square that does not exist:

	+ RoleBinding	+ ClusterRoleBinding
Role	Works — permissions in that namespace	**Invalid.** The API rejects it.
ClusterRole	**The useful one** — define once, bind where it applies	Cluster-wide, including namespaces not yet created

5. Internal architecture

The RBAC authorizer is one of several authorization modes in the API server (`--authorization-mode=Node,RBAC`). Modes are consulted in order; the first to return "allow" or "deny" wins, and RBAC never returns "deny" — only "no opinion". This is why adding a second authorizer can grant, but never revoke.

Roles and bindings are cached and indexed in memory, so authorisation does not hit etcd per request.

6. Component interactions

`kubectl auth can-i` issues a **SubjectAccessReview** to the API server. That is the same authorisation path a real request takes, with the same cache and the same rules. It is not a simulation of your policy — it *is* your policy.

```
kubectl auth can-i get secrets -n axispay-core --as=auditor@axis.example
# no
```

`--as` requires impersonation permission, which is itself a significant grant. On a locked-down cluster it will fail, and that is a control worth knowing about.

7. Enterprise example

A financial services platform team passed an access review on the strength of "nobody has `get` on `secrets` in the payments namespace". Eleven engineers had `pods/exec` in the same namespace. The JWT signing key was an environment variable in the auth service.

```
kubectl exec -n axispay-edge deploy/auth-service -- printenv JWT_SIGNING_KEY
```

No `secrets` grant required. The review had checked a resource rather than a path.

8. Real-world analogy

RBAC is a building where each door has a list of who may open it. There is no "everyone except Dave" list — if Dave should not get in, his name is simply not on the door.

`pods/exec` is the cleaner's master key. It does not appear on any door's list, and it opens the room where the safe combination is written on a whiteboard.

9. Best practices

- **Prove every grant.** `kubectl auth can-i` output is the deliverable; the YAML is the implementation.
- **Prove the denials too.** A list of six `no` responses is what an access review actually needs.
- **ClusterRole + RoleBinding** for anything you will bind in more than one namespace.
- **RoleBinding by default; ClusterRoleBinding only for genuinely cluster-scoped resources** — nodes, storage classes, CRDs.
- **Annotate roles that grant more than they appear to.** `pods/exec` is equivalent to reading every Secret in the namespace. Say so on the object.
- **Audit the aggregate, not the intent:** `kubectl auth can-i --list` shows the union of every binding.

10. Common mistakes

Mistake	What happens
Granting `*` on `*` "temporarily"	It survives every subsequent review. Nobody removes a grant that has not visibly broken anything.
Reviewing `secrets` and stopping	`pods/exec`, `pods/portforward` and `create` on pods all yield the same access.
ClusterRoleBinding by reflex	Grants in every namespace, including ones nobody has created yet.
Granting `pods` and expecting `pods/log`	Subresources are separate. The user gets a confusing partial denial.
Assuming a permission can be revoked from one user	There is no deny. Find the binding that grants it.

11. Security considerations

The paths to a Secret, in order of how often they are missed:

1. `get` / `list` on `secrets` — the obvious one.
2. `create` on `pods/exec` — read any Secret consumed by any pod in the namespace.
3. `create` on `pods` — mount any Secret into a pod you create.
4. `create` on `pods/portforward` — reach a service that exposes it.
5. `escalate` or `bind` on roles — grant yourself the first one.
6. `impersonate` on users or groups — become someone who has it.

An RBAC review that checks item 1 and stops has audited a resource, not a path.

12. Performance considerations

Authorisation is in-memory and does not measurably affect API latency. Very large numbers of bindings (tens of thousands) increase memory and cache-rebuild time, which is a scale problem rather than a latency one.

13. High availability

RBAC objects live in etcd and are cached by every API server instance. There is no separate service and nothing extra to make highly available.

Do keep one break-glass credential — a certificate-based cluster-admin — outside your normal identity provider. If OIDC is down, everything else is too.

14. Disaster recovery

Roles and bindings are plain objects; keep them in Git and they restore with everything else. What does *not* restore from your cluster backup is the identity provider mapping — the

group names in your bindings mean nothing without the IdP that issues them. Document that dependency.

15. Monitoring

The API server audit log records the authorisation decision for every request. The two things worth alerting on:

- creation of any ClusterRoleBinding (rare, and always significant)
- any use of `impersonate`

```
sum by (code) (rate(apiserver_request_total{code="403"}[5m]))
```

A rising 403 rate usually means a workload was deployed with the wrong ServiceAccount — cheap to detect, otherwise a confusing partial failure.

16. Troubleshooting

Symptom	Cause	Command
`forbidden: User cannot list X`	No rule grants it	`kubectl auth can-i --list --as=<user> -n <ns>`
Permission appears from nowhere	Another binding grants it	`kubectl get rolebinding,clusterrolebinding -A -o wide | grep <subject>`
`pods/log` denied, `pods` allowed	Subresources are separate	Grant `pods/log` explicitly
`cannot impersonate`	`--as` requires permission	Expected on a real cluster
Works for you, not for the pipeline	You are cluster-admin	Always test with `--as`

Interview questions

1. **How do you revoke a permission in Kubernetes RBAC?** — You cannot. There is no deny; you remove or narrow the binding that grants it.
2. **Give a case for ClusterRole + RoleBinding.** — One permission set bound in several namespaces. Define once, avoid drift.
3. **Can a ClusterRoleBinding reference a Role?** — No. The API rejects it.
4. **Why is `pods/exec` more dangerous than it looks?** — It reads every Secret consumed by those pods, with no `secrets` grant.
5. **How do you prove an auditor cannot read Secrets?** — `kubectl auth can-i get secrets -n <ns> --as=<user>` returning `no`, for every relevant namespace.
6. **What is `escalate` for?** — It permits granting permissions you do not hold yourself. Without it, RBAC prevents privilege escalation by construction.

5.3 Helm

1. What it is

A package manager for Kubernetes. A **chart** is a directory: `Chart.yaml` (metadata), `values.yaml` (data), and `templates/` (Go templates rendered against those values). `helm install` renders the templates and applies the result. A **release** is one installation of a chart, with a revision history.

2. Why it exists

Two problems. First, parameterisation: the same platform must exist in dev, staging and production with different replica counts and hostnames, and copy-pasted YAML directories drift within weeks. Second, lifecycle: `kubectl apply -f dir/` has no concept of "the set of objects that belong together", so it cannot tell you what a change will do, cannot remove objects you deleted from the directory, and cannot roll back.

3. The business problem

AxisPay needs a second environment for merchant integration and a third for acquirer certification. The runbook for building one is four days of this course. Worse, the three will drift: someone bumps a replica count in staging and not production, someone fixes a probe in production and not staging, and in six weeks nobody can say what the difference is.

4. How it works

Rendering is text substitution. That is not a simplification — Go's `text/template` has no idea it is producing YAML:

```
{{- range $name, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ $name }}
spec:
  replicas: {{ $svc.replicas }}
{{- end }}
{{- end }}
```

Whitespace control matters because of that. `{{-` strips preceding whitespace including newlines; `-}}` strips following whitespace. A missing dash produces a blank line; an extra one joins two lines together and breaks the YAML.

The commands worth knowing:

```
helm template axispay ./charts/axispay          # render only — do this first, always
helm lint ./charts/axispay -f values-prod.yaml  # syntax and conventions
helm upgrade --install axispay ./charts/axispay --atomic --timeout 10m
helm history axispay
helm rollback axispay 3
helm diff upgrade axispay ./charts/axispay      # plugin — shows what WOULD change
```

5. Internal architecture

Helm 3 has **no server-side component**. Helm 2's Tiller ran in-cluster with broad permissions and was a standing security problem; it is gone. The client renders, diffs and applies using your own kubeconfig and your own RBAC.

A release is stored as a **Secret in the release namespace**, one per revision, containing a gzipped, base64-encoded copy of the rendered manifests:

```
kubectl get secret -n default -l owner=helm
```

Three consequences follow:

- `helm rollback` re-applies a **stored copy**. It does not re-render from your repository, so it works even if the chart source has changed.
- Deleting those Secrets loses your release history permanently.
- Between commands, Helm is not running. **Nothing watches your chart.**

6. Component interactions

```
helm upgrade
   → render templates against merged values
   → fetch the last release Secret
   → three-way merge: last release, live state, new render
   → PATCH / CREATE / DELETE against the API server
   → write a new release Secret
   → (--atomic) on failure, roll back and write another
```

The three-way merge is why Helm mostly tolerates a field some other controller owns — but "mostly" is why the chart omits `.spec.replicas` for HPA-managed Deployments rather than relying on it.

7. Enterprise example

A payments platform put `app.kubernetes.io/version` in every Deployment's `.spec.selector`. Install worked. Three patch releases worked, because the version had not changed. The first minor version bump failed:

```
Error: UPGRADE FAILED: cannot patch "payment-service" with kind Deployment:
Deployment.apps "payment-service" is invalid: spec.selector: field is immutable
```

The only fix is deleting the Deployment — in production, during the failed release. The defect had been in the chart for four months.

8. Real-world analogy

A chart is a **mail merge**. `values.yaml` is the spreadsheet, `templates/` is the letter, and the output is a stack of letters. The mail merge does not know it is producing letters; it substitutes text. If your spreadsheet has a comma in the wrong place, you get a malformed letter, not an error.

`helm template` is printing one to check before you post two thousand.

9. Best practices

- **`helm template` before every install and every upgrade.** Three seconds.
- **`--atomic` on every upgrade.** It rolls back automatically on failure. There is no case for omitting it.
- **Keep `.spec.selector` to identity labels only.** Never chart version, app version or environment.
- **Omit `replicas` when an HPA owns the workload**, or every upgrade fights the autoscaler.
- **`--set` for experiments, never for state.** A change that lives in shell history is drift by construction.
- **A `NOTES.txt` that contains a real smoke test**, not "deployed successfully".
- **Pin the image tag.** You cannot roll back to a tag that moves.

10. Common mistakes

Mistake	What happens
Volatile labels in `.spec.selector`	The second release with a version change fails, immovably
Pinning `replicas` on an HPA-managed Deployment	Every upgrade resets the replica count, sometimes mid-spike
Upgrading without `--atomic`	A failed upgrade leaves half the pods on each version
`--set` as a permanent mechanism	Nobody can reconstruct what is deployed
`latest` as the tag	Rollback is meaningless
Deleting release Secrets to "clean up"	History gone; rollback impossible

11. Security considerations

Helm 3 uses your credentials and your RBAC, so it can do exactly what you can do — no more. That is the main security improvement over Helm 2.

Charts are code. `helm install` on a chart from an unknown repository runs its templates and applies whatever they produce, including RBAC objects. Charts can be signed (`helm package --sign`, `helm verify`); almost nobody does this, and it is worth doing for anything you did not write.

Values files hold configuration, not secrets. Passwords belong in a Secret managed separately — a sealed-secrets controller, an external secrets operator, or your cloud's secret manager.

12. Performance considerations

Rendering is milliseconds. The slow part of an upgrade is waiting for rollouts, which is Kubernetes, not Helm.

`--wait` blocks until resources report ready, and its usefulness depends entirely on your readiness probes being honest. A chart that "installs successfully" with broken readiness probes has told you nothing.

13. High availability

Helm has nothing to make highly available — it is a CLI. What matters is that the release Secrets live in the cluster, so they are as durable as etcd.

14. Disaster recovery

The chart is in Git; the release state is in etcd. Rebuilding a cluster means `helm install` again, not restoring release Secrets. Do not treat the release history as a backup — it records what you applied, not your data.

15. Monitoring

Helm emits no metrics. What you monitor is the result: are the Deployments at the expected version, is the release `deployed` rather than `failed`?

```
# rollouts that have not converged
kube_deployment_status_observed_generation != kube_deployment_metadata_generation
```

In CI, `helm diff upgrade` returning non-empty against your production values is a drift alarm.

16. Troubleshooting

Symptom	Cause	Command
`cannot re-use a name that is still in use`	Objects exist from `kubectl apply`	Delete them, or `--force`
`field is immutable`	Selector labels changed	Delete the Deployment — so prevent the cause
Release stuck `pending-upgrade`	An upgrade was interrupted	`helm rollback`
`helm lint` **clean but objects wrong**	Lint checks syntax, not semantics	Write assertions — see `check-helm-chart.py`
Renders locally, fails on apply	An admission controller rejects it	`helm template ... | kubectl apply --dry-run=server -f -`

Interview questions

1. **Where does Helm 3 store release state, and why does that matter?** — A Secret per revision in the release namespace, holding gzipped rendered manifests. Rollback re-applies a stored copy rather than re-rendering.
2. **Why must** `.spec.selector` **contain only stable labels?** — It is immutable after creation; a version label makes the next version bump fail.
3. **What does** `--atomic` **do?** — Rolls back automatically if the release does not become ready within the timeout, avoiding a half-applied state.
4. **How do Helm and an HPA conflict, and how do you prevent it?** — If the chart sets `replicas`, every upgrade resets the count. Omit the field when an HPA owns it.
5. **Is Helm a controller?** — No. Nothing watches the chart; between commands Helm is not running.
6. **What does** `helm lint` **not check?** — Semantics. It has no opinion about whether your liveness probe points at the same endpoint as your readiness probe.

5.4 Environment promotion

1. What it is

Running the same artefact in several environments, with the differences expressed as data and reviewable in a diff.

2. Why it exists

Because the alternative is three directories of YAML that were identical once, and nobody can now say what the differences are or which were intentional.

3. The business problem

A payment that works in staging fails in production. Both were "deployed from the same chart". Four hours of investigation later, staging has `maxUnavailable: 1` and production has `0`, set during an incident eight months ago and never restored. Nobody was careless — there was simply no artefact stating the intended difference, so there was nothing to check drift against.

4. How it works

One chart, several values files, merged deeply with later files winning:

```
helm upgrade --install axispay ./charts/axispay -f values-prod.yaml
```

The rule: structure identical, numbers different. A staging environment that differs structurally from production is testing a system that does not exist.

What legitimately differs

Setting	dev	prod	Why
`replicas`	1	3–6	Nobody depends on dev availability
`podDisruptionBudget`	off	on	A PDB on one replica blocks every drain
`logLevel`	debug	info	Debug logs would emit request bodies
`ingress`	off	on	Port-forward is faster in dev
`maxUnavailable`	1	0	With one replica, surge alone cannot progress

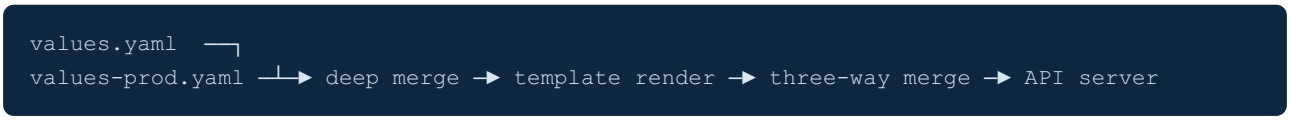
What never differs

Setting	Every environment	Why
`networkPolicy.enabled`	`true`	A policy you only enable in production is one you first test in production
`podSecurity.enforce`	`restricted`	Same argument
`serviceAccount.automountToken`	`false`	Same argument

5. Internal architecture

Helm's value merging is a deep merge (`mergeMaps`): maps merge recursively, scalars and lists are replaced wholesale, and an explicit `null` deletes a key. **Lists replace rather than append**, which surprises people who expect their environment file to add one item to a list of five.

6. Component interactions



`--set` is applied after all files, which is exactly why it is fine for experiments and unacceptable for state.

7. Enterprise example

A bank ran a "staging" cluster with one replica per service and no PodDisruptionBudgets, because it was cheaper. Every release passed staging. The first production release after adding PDBs hung for forty minutes: with `maxUnavailable: 0` and a PDB that could not be satisfied, the rollout could make no progress and staging had never exercised either setting.

Staging's numbers can shrink. Its shape cannot.

8. Real-world analogy

A car crash-test programme. The test vehicle is the same model, the same structure, the same crumple zones — it just has instruments instead of upholstery. What it is *not* is a different car that happens to be the same colour.

9. Best practices

- **Same object kinds in every environment.** Diff them in CI; fail the merge if they differ.
- **State every difference in the values file, with the reason in a comment.**
- **Never relax security per environment.**
- **`helm diff upgrade` in CI** against production values, as a drift alarm.
- **Promote by changing the values file**, through review — not by `--set` in a terminal.

10. Common mistakes

Mistake	What happens
Staging structurally simpler than production	Rollout, disruption and policy behaviour are never tested
Security relaxed in dev	The first real test of your NetworkPolicy is production
`--set` used for durable changes	Nobody can reconstruct what is deployed
Expecting list merging	Lists replace. Your one item replaces their five
Fixing drift by editing the cluster	It reverts at the next upgrade

11. Security considerations

The values file is the audit trail for a security posture. `grep networkPolicy charts/axispay/values-*.yaml` should return `true` for every environment, and that one-line check is a genuine control.

Nothing secret belongs in a values file. They are read by everyone with repository access.

12. Performance considerations

Not a performance topic — except that production resource requests are a values-file decision, and the wrong ones there are the difference between a cluster that schedules and one that does not.

13. High availability

Anti-affinity, PodDisruptionBudgets, replica counts and topology spread are all values-file settings. Dev turns them off because they are meaningless at one replica; production turns them on. That difference is legitimate and must be written down.

14. Disaster recovery

A DR environment is one more values file. The useful discipline is to keep the alerting **on** there, at full strength — so you learn it is broken before you need it, rather than during the failover.

15. Monitoring

Alert on drift:

```
helm diff upgrade axispay ./charts/axispay -f values-prod.yaml
```

Non-empty output in CI means the cluster no longer matches the repository. Someone edited production.

16. Troubleshooting

Symptom	Cause	Command
Works in staging, fails in production	A structural difference	`diff <(helm template -f staging) <(helm template -f prod)`
Diff shows changes you did not make	A controller owns the field	HPA replicas, and Kubernetes' own defaults
Values file appears ignored	Wrong path, or overridden by a later `-f`	Order matters; last wins
List has the wrong contents	Lists replace, not append	State the whole list

Interview questions

1. **Why must staging match production structurally?** — Otherwise you never test rollout, disruption or policy behaviour, which is what staging is for.
2. **Name a setting that should never be relaxed in dev.** — NetworkPolicy. A policy you only enable in production is one you first test in production.
3. **How do Helm values merge?** — Deep merge; later files win; lists replace; explicit null deletes.
4. **How would you detect that someone edited production by hand?** — `helm diff upgrade` in CI, on a schedule.
5. **Why does the chart omit `replicas` for HPA-managed workloads?** — So an upgrade does not reset the autoscaler's decision.

5.5 Metrics, PromQL and dashboards

1. What it is

Prometheus scrapes HTTP endpoints exposing text-format metrics, stores them as time series, and answers PromQL queries. Grafana draws the results. In this platform the ServiceMonitor CRD, owned by the Prometheus Operator, tells Prometheus what to scrape.

2. Why it exists

Because operating a system you cannot measure is guesswork, and because "is it working?" needs a numeric answer that two teams can agree on.

The **pull** model is a deliberate choice. Prometheus fetches from your service rather than your service pushing to Prometheus. That gives you a free liveness signal (a target that cannot be scraped is visibly down), it removes any need for services to know where the monitoring lives, and a monitoring outage cannot back-pressure your application.

3. The business problem

AxisPay has an SLO: 99.5% availability, 300 ms p99 on the payment API. Last quarter the platform team says it was met. The merchant integration team has a spreadsheet of failed calls and disagrees. Neither can settle it, because "availability" was never defined in a way a machine could compute.

4. How it works

Four metric types. Counter (only increases; use `rate()`), Gauge (goes up and down), Histogram (bucketed observations, enables quantiles), Summary (client-computed quantiles — cannot be aggregated across pods, so prefer histograms).

The four golden signals, as queries:

```
# TRAFFIC
sum by (service) (rate(axispay_http_requests_total[5m]))

# ERRORS — only 5xx. A 409 fraud decline is the system working.
sum(rate(axispay_http_requests_total{status=~"5.."}[5m]))
  / sum(rate(axispay_http_requests_total[5m]))

# LATENCY — a histogram, never an average
histogram_quantile(0.99, sum by (le) (
  rate(axispay_http_request_duration_seconds_bucket{service="payment-service"}[5m])))

# SATURATION — against the REQUEST, which is what the HPA divides by
sum by (pod) (rate(container_cpu_usage_seconds_total{namespace="axispay-core"}[5m]))
  / sum by (pod) (kube_pod_container_resource_requests{namespace="axispay-
core",resource="cpu"})
```

Why not the average. If 99 requests take 10 ms and one takes 3 s, the mean is 40 ms — and one merchant in a hundred waited three seconds. The SLO is written on the p99 because that is the customer you are about to lose.

5. Internal architecture

Prometheus stores samples in a local TSDB: a 2-hour write-ahead-logged head block, compacted into immutable blocks on disk. It is **not** clustered and not a long-term store. Federation, Thanos or Mimir exist for that.

A **series** is a unique combination of metric name and label values. Memory is roughly proportional to the number of active series, which is why label cardinality is the central design constraint.

The **Prometheus Operator** watches ServiceMonitor, PodMonitor, PrometheusRule and AlertmanagerConfig objects, generates a configuration, writes it to a Secret and triggers a reload. You never edit `prometheus.yml`.

6. Component interactions

```
app: Counter.inc()
  → /metrics (text endpoint)
  → Prometheus scrapes every 15-30s
       ↑ ServiceMonitor → operator → scrape config
  → TSDB
  → PromQL → Grafana panels
  → PrometheusRule → alerts → Alertmanager
```

The ServiceMonitor selects the **Service**, and Prometheus discovers the endpoints behind it. An unready pod therefore is not scraped — readiness gating for free, and a starting pod does not drag your averages around.

7. Enterprise example

A team added `payment_id` as a Prometheus label to make debugging easier. Within four hours Prometheus was consuming 60 GB and being OOMKilled. Every payment created a new series, and every series persisted in memory for the retention window.

The debugging information they wanted belonged in a log line, where high-cardinality data is free.

8. Real-world analogy

Metrics are the instrument panel: speed, fuel, temperature — a small fixed set, always present, cheap to read. Logs are the dashcam: high detail, expensive, consulted after something happens.

Putting a payment ID in a metric is asking the speedometer to display every registration number it passes.

9. Best practices

- **Bounded labels only.** service, method, route, status, currency. Never IDs.
- **Route templates, not raw paths.** `/api/v1/payments/{id}`, not `/api/v1/payments/pay_7Kq2`.
- **Place histogram buckets around your SLO threshold** so the quantile is accurate where it matters.
- **Alert on symptoms; graph causes.**
- **Dashboards as ConfigMaps**, generated and reviewed. Not clicked and exported.
- **Test your PromQL.** A misspelled metric evaluates to an empty vector and the alert silently never fires.

10. Common mistakes

Mistake	What happens
High-cardinality labels	One series per request; Prometheus OOMs
`rate()` on a gauge	Meaningless output, no error
Averaging a latency	Hides the tail entirely
Counting 4xx as errors	The error budget burns on correct behaviour
ServiceMonitor without the `release` label	Target absent — not down — with no diagnostic
Editing dashboards in the UI	Lost at the next restart, never reviewable

11. Security considerations

`/metrics` reveals internal structure: route names, dependency names, error rates. In this platform it is on the pod port and reachable only from the observability namespace, enforced by NetworkPolicy.

Never put a secret, a token or a customer identifier in a label. Metrics are widely readable by design.

12. Performance considerations

Scraping costs the application almost nothing — serialising a few hundred series. The cost is on the Prometheus side and scales with **active series**, not with request volume.

`metricRelabelings` that drop noisy series (`python_gc_*`, `process_*`) at ingestion is the cheapest optimisation available.

13. High availability

Run two Prometheus instances scraping the same targets. They will not agree exactly, which is fine for alerting and unhelpful for long-term reporting — which is what Thanos and Mimir address.

Alertmanager clusters properly: run three, and they deduplicate notifications between themselves.

14. Disaster recovery

Metrics are usually not backed up. Six hours of retention on a training cluster and fifteen days in production are both operational data, not records. If a metric matters for reporting or compliance, it belongs in a long-term store, not in Prometheus.

Rules and dashboards are in Git. Those are the parts that must survive.

15. Monitoring the monitoring

```
up{job=~"axispay.*"} == 0                          # a target is down
prometheus_tsdb_head_series                         # active series — watch it grow
rate(prometheus_target_scrapes_exceeded_sample_limit_total[5m]) > 0
```

The last one catches a cardinality explosion before it becomes an outage.

16. Troubleshooting

Symptom	Cause	Command
Target missing entirely	ServiceMonitor not selected	Check `release: kube-prometheus-stack`
Target down, connection refused	Wrong port name, or pod unready	ServiceMonitor uses the port NAME
Target down, deadline exceeded	Timeout, or NetworkPolicy	Check `allow-prometheus-scrape`
Query returns nothing	That label does not exist on that metric	Read `/metrics` — you cannot query what you did not emit
`histogram_quantile` returns NaN	No observations in the window	Generate traffic or widen the range

Interview questions

1. **Why does Prometheus pull rather than push?** — Free liveness signal, no service needs to know where monitoring lives, and a monitoring outage cannot back-pressure the app.
2. **Why is the average latency misleading?** — It is dominated by the fast majority and hides the tail the SLO is written on.
3. **What is a series, and why does cardinality matter?** — A unique metric-name/label-value combination. Memory scales with active series.
4. **A target is missing rather than down. What does that mean?** — Prometheus never selected it — usually a ServiceMonitor label. Down means it tried and failed.

5. **Why does a ServiceMonitor select a Service rather than pods?** — Endpoint discovery means unready pods are not scraped.
6. **Where should high-cardinality data live?** — In log lines, queried by label selector plus a body filter.

5.6 Logs and alerting

1. What it is

Loki stores logs indexed by labels; the body is compressed and scanned at query time. **Alloy** is the collector: a DaemonSet reading `/var/log/pods` on each node, relabelling and shipping. **Alertmanager** receives firing alerts from Prometheus and decides who is notified, how often, and whether an alert should be suppressed.

2. Why it exists

Loki exists because indexing the full text of every log line — the Elasticsearch model — is expensive at scale, and most queries start by narrowing to a service and a time range anyway. Indexing only labels makes ingestion cheap; the trade is that content filtering is a scan.

Alertmanager exists because Prometheus deliberately does not know about teams, schedules, escalation or deduplication. Prometheus decides *whether* something is wrong. Alertmanager decides *who* hears about it.

3. The business problem

02:14 on a Saturday. p99 latency spikes to 4 seconds for ninety seconds, then recovers. No alert fires — `for: 10m` did its job and nobody was woken for something that healed itself.

Monday, a merchant complains: payment reference `AXP-4471-ZA` took four seconds and their checkout timed out. They have the reference. Seven services touched that payment. Which one was slow?

4. How it works

LogQL is a label selector followed by optional filters:

```
{namespace="axispay-core", service="payment-service"}          # index lookup
{namespace=~"axispay-.*"} | json | correlation_id="7f3a9c21"   # then a scan
{namespace=~"axispay-.*"} | json | duration_ms > 500           # a scan too
```

The label selector is not optional. It is what makes the query finite.

Alertmanager's four mechanisms:

Mechanism	Problem it solves
Grouping	One bad node takes out eight pods — one notification, not eight
Inhibition	Zero endpoints implies high errors and high latency — one page, not three
Throttling	`repeatInterval` stops a four-hour incident paging every thirty seconds
Routing	Risk alerts to risk, reconciliation to finance-ops

5. Internal architecture

Loki stores an index (TSDB, in current versions) plus compressed chunks. A **stream** is a unique set of label values; each has its own chunks. Stream count is the scaling constraint — the same shape of problem as Prometheus series count, with the same fix.

Alertmanager holds a notification log and a silence store, gossiped between replicas so a three-node cluster deduplicates correctly.

6. Component interactions

```
pod → stdout → /var/log/pods on the node
    → Alloy (DaemonSet, one per node)
         labels: namespace, service, pod, container, node   ← bounded
         body:   correlation_id, payment_id, duration_ms    ← unbounded
    → Loki → Grafana Explore

Prometheus → alert firing → Alertmanager
    → group → inhibit → throttle → route → receiver
```

Logging to **stdout** is the contract that makes this work. A container writing to `/var/log/app.log` inside itself produces logs nobody can read and that vanish when the pod restarts.

7. Enterprise example

A team promoted `trace_id` to a Loki label to make traces easy to find. It worked beautifully in a test environment with forty requests. In production at 200 requests per second it created 17 million streams in a day. Loki's ingesters ran out of memory, and the log platform went down during the incident it had been bought for.

The fix was one line of relabelling config, and the outage was six hours.

8. Real-world analogy

Loki is a filing cabinet with drawer labels. The drawers say "payments, August". They do not say "every document mentioning invoice 4471". You open the right drawer and read.

Giving each document its own drawer is what a high-cardinality label does.

9. Best practices

- **Structured JSON on stdout**, one object per line, always with a correlation ID.
- **Bounded labels only**: namespace, service, pod, container, node.
- **Always start a query with a label selector.**
- **Exclude probe endpoints from access logs** — the kubelet is not traffic.
- **Every alert has `for:`, a severity, a summary and a runbook link.**
- **Inhibition rules for symptom chains**, so one fault produces one page.
- **`sendResolved: true`**, or the channel fills with problems and never with endings.

10. Common mistakes

Mistake	What happens
High-cardinality Loki label	Stream explosion; the log platform dies
Query with no label selector	Loki reads everything it has
Logging to a file in the container	Nobody can read it; it vanishes on restart
Alert with no `for:`	A single slow scrape pages someone
No inhibition	One node produces eight pages, and the channel gets muted
Alert with no runbook	The page starts with twenty minutes of reading

11. Security considerations

Logs frequently contain more than intended. AxisPay logs a card **token** and a last-four, never a PAN — and `scripts/validate` greps the seed data to prove it.

`LOG_LEVEL=debug` in production is a data-protection incident waiting to happen: debug logging typically emits request bodies.

Loki has no authentication in single-tenant mode. Reach it through Grafana, and put Grafana behind SSO.

12. Performance considerations

Ingest cost scales with volume; query cost scales with the number of streams and bytes scanned. A tight label selector on a narrow time range is fast; `{namespace=~".+"}` over seven days is not.

Dropping probe-endpoint access logs at the application removes the single largest source of volume in this platform.

13. High availability

Single-binary Loki has no redundancy, which is the correct choice for a classroom. Production uses the distributed mode with a replication factor and object storage.

Alertmanager should always be at least three replicas — losing your alert delivery during an incident is a particularly poor failure mode.

14. Disaster recovery

Logs are usually not backed up; retention is the policy. What must survive is the **configuration** — Alloy's relabelling rules, Loki's limits, the Alertmanager routing tree — and all of it is in Git.

If a log is a compliance record, it belongs in an archive with a retention policy, not only in Loki.

15. Monitoring the logging

```
loki_ingester_streams_created_total          # the cardinality alarm
rate(loki_request_duration_seconds_count{status_code=~"5.."}[5m])
alertmanager_notifications_failed_total      # alerts that were never delivered
```

The last one is the quietest failure in the whole stack: alerts fire, nobody is notified, and nothing tells you.

16. Troubleshooting

Symptom	Cause	Command
No logs at all	Alloy not running	It needs `privileged` PSA for hostPath
Logs present, `| json` empty	Not JSON, or the format changed	`kubectl logs ... | jq .`
Query times out	No label selector, or too wide a range	Narrow both
Alert firing, nothing received	Route matchers do not match the labels	Compare the alert's labels to the route
Alert pending forever	`for:` has not elapsed	That is the design
Two teams paged for one fault	Inhibit rule labels do not match	`equal:` must name labels on both alerts

Interview questions

1. **Why does Loki index labels and not content?** — Cheap ingestion; most queries narrow by service and time anyway. The trade is that content filtering is a scan.
2. **What happens if you make `request_id` a Loki label?** — One stream per request; the ingesters run out of memory.
3. **What does Alertmanager do that Prometheus does not?** — Grouping, inhibition, throttling and routing — everything about *who* is notified.
4. **Give a case for an inhibit rule.** — Zero ready endpoints implies high error rate and high latency; suppress the derived symptoms.
5. **Why must containers log to stdout?** — The kubelet captures it; `kubectl logs` reads it; collectors ship it. A file inside the container is unreachable and impermanent.
6. **How would you find every log line for one payment across seven services?** — Query the correlation ID in the body with a namespace label selector: `{namespace=~"axispay-.*"} | json | correlation_id="..."`.

5.7 Cluster upgrades and change management

1. What it is

Upgrading Kubernetes itself: the control plane, then the kubelets, then the client tools. On a self-managed cluster this is `kubeadm upgrade`; on a managed one it is a console button that runs the same sequence.

This is the one module of the week where you will **not** run the procedure. Upgrading a Minikube cluster mid-course would end the course. What you get instead is the ordering rules, the constraints, and the one operation you have already practised — drain — which is where the risk actually lives.

2. Why it exists

Kubernetes releases a minor version roughly every four months and supports each for about fourteen. You cannot opt out: staying still means running an unsupported control plane, and the upgrade you defer for a year becomes four upgrades you must do in sequence.

3. The business problem

AxisPay's cluster is two minor versions behind. A CVE lands in a component patched only in the current release. The upgrade must happen inside a change window, with merchant traffic continuing, and the platform team has never done one on this cluster.

4. How it works

Version skew is the rule that decides the ordering. In current Kubernetes:

Component	May be behind the API server by
kubelet	up to **three** minor versions
kube-proxy	up to three minor versions
controller-manager, scheduler	one minor version
`kubectl`	one minor version either way

Because the kubelet may lag the API server but must never *lead* it, the control plane goes first. Always.

The sequence:

```
# 1. control plane, first node
kubeadm upgrade plan                        # tells you what is available and what it will
do
kubeadm upgrade apply v1.36.2

# 2. that node's kubelet — drain it first
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
apt-get install -y kubelet=1.36.2-* kubectl=1.36.2-*
systemctl daemon-reload && systemctl restart kubelet
kubectl uncordon <node>

# 3. remaining control-plane nodes
kubeadm upgrade node

# 4. worker nodes, ONE AT A TIME
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
# upgrade kubeadm, kubeadm upgrade node, upgrade kubelet, restart
kubectl uncordon <node>
```

One minor version at a time. You cannot jump from 1.33 to 1.36. Each step is its own upgrade, and each is a separate change record.

5. Internal architecture

`kubeadm upgrade apply` renews the control-plane component manifests in `/etc/kubernetes/manifests`, which the kubelet is watching as static pods — so replacing the file restarts the component. It also renews the control-plane certificates, which is a genuinely useful side effect: certificates expire after a year, and clusters that are never upgraded are the ones that die of certificate expiry.

etcd is upgraded as part of the same sequence, and its data directory is backed up first — but only on the node being upgraded. That is not a backup strategy.

6. Component interactions

```
kubeadm upgrade apply
   → pre-flight checks (skew, health, disk)
   → etcd backup on this node
   → replace static pod manifests: apiserver, controller-manager, scheduler
   → kubelet restarts them
   → renew control-plane certificates
   → update the kubelet config in the cluster ConfigMap

per node:  drain → upgrade kubelet → uncordon
```

The drain is where your Day 4 work pays off. Every workload with a PodDisruptionBudget constrains the drain; every workload with `maxUnavailable: 0` and a preStop hook survives it without dropping a request. An upgrade of a platform without PDBs is a sequence of small outages nobody planned.

7. Enterprise example

A bank ran a four-node upgrade with `--force` on the drain because two nodes were taking too long. The drain was slow precisely because a PodDisruptionBudget was doing its job: the second replica of a stateful service had not become ready. `--force` deleted it anyway, and the service was down for eleven minutes with both replicas gone.

The PDB was not the problem. It was the only thing that had been telling them the truth.

8. Real-world analogy

Refurbishing a hotel one floor at a time while guests stay. You do not close the hotel; you close a floor, move the guests, do the work, reopen it. The rule that stops you closing two floors at once is the PodDisruptionBudget — and overriding it because you are behind schedule is how you end up with guests in the car park.

9. Best practices

- **Read the release notes for every minor version you pass through.** Removed APIs are the usual cause of a failed upgrade, and `kubectl` will not warn you.
- **Check deprecated API usage first.** `kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis` shows what in your cluster still calls something scheduled for removal.
- **Upgrade a non-production cluster of the same version first.** Not a smaller one — the same version, with the same workloads.
- **Never `--force` a drain.** If it is slow, find out why. It is usually telling you something true.
- **One node at a time,** and validate between each: `make validate-day5` or its production equivalent.
- **Have the control plane backed up** — etcd snapshot — before you start, not just kubeadm's local copy.

10. Common mistakes

Mistake	What happens
Upgrading kubelets before the control plane	Kubelets ahead of the API server are unsupported and behave unpredictably
Skipping a minor version	kubeadm refuses; if you force the packages, you get an unsupported skew
`--force` on a slow drain	You delete the pod the PDB was protecting
Not checking removed APIs	Workloads that applied fine last week are rejected after the upgrade
Treating kubeadm's etcd backup as the backup	It is on the node being upgraded, which is the node most likely to be lost

11. Security considerations

Upgrades are frequently *why* you are upgrading — a CVE in the API server or the kubelet. Note that a control-plane upgrade does not patch your **workload** images; those are a separate pipeline, and conflating the two leaves a cluster whose components are current and whose containers are two years old.

Certificate renewal during upgrade is a security benefit worth naming: a cluster that is upgraded regularly does not die of expired certificates.

12. Performance considerations

The API server is briefly unavailable while its static pod restarts — seconds on a single-control-plane cluster, and effectively zero on a multi-node control plane behind a load balancer. **Running workloads are unaffected**: the kubelet keeps containers running when it cannot reach the API server. What stops is the *control loop* — no scheduling, no scaling, no rollouts.

13. High availability

Three control-plane nodes and a load balancer in front of them makes the upgrade genuinely non-disruptive to the API. With one control-plane node, plan for a short API outage and do not schedule the upgrade during a release.

14. Disaster recovery

Take an etcd snapshot before you begin and copy it off the node:

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-pre-upgrade.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

A control-plane upgrade cannot be rolled back. That is the constraint to state at the change board: forward-only, with a restore-from-snapshot as the recovery path rather than a downgrade.

15. Monitoring

```
count by (kubelet_version) (kubernetes_build_info{job="kubelet"})   # the skew, live
kube_node_status_condition{condition="Ready",status="true"} == 0    # a node that did not
come back
apiserver_requested_deprecated_apis                                  # what will break at
the next upgrade
```

The last one is the useful one *between* upgrades. It tells you months ahead which of your manifests will stop working.

16. Troubleshooting

Symptom	Cause	Command
`kubeadm upgrade plan` refuses	Skew too large, or an unhealthy component	Read the pre-flight output; upgrade one minor version at a time
Drain hangs	A PDB is being satisfied — correctly	`kubectl get pdb -A` ; fix the workload, do not force
Node `NotReady` after upgrade	kubelet failed to start	`journalctl -u kubelet -n 50`
Workloads rejected after upgrade	A removed API version	`kubectl api-resources` ; update the manifests
API server unreachable mid-upgrade	Expected on a single control-plane node	Wait; running workloads are unaffected

Interview questions

1. **Why does the control plane get upgraded before the kubelets?** — The kubelet may lag the API server by up to three minor versions but must never lead it.
2. **How many minor versions can you skip?** — None. One at a time, each its own change.
3. **A drain is taking twenty minutes. What do you do?** — Find out which PodDisruptionBudget is constraining it and why the replacement is not ready. Not `--force`.
4. **What happens to running pods while the API server is down?** — They keep running. The kubelet does not need the API server to keep containers alive. What stops is scheduling, scaling and rollouts.
5. **Can you roll back a control-plane upgrade?** — No. The recovery path is a restore from an etcd snapshot, which is why you take one first.
6. **What tells you, months in advance, that an upgrade will break your workloads?** — `apiserver_requested_deprecated_apis`.

5.8 Supply Chain Security for Java Microservices

1. What it is

Supply chain security answers three questions about a workload before it runs:

Question	Control
What is in the image?	SBOM
Who built it?	signing and provenance
Was it changed after build?	signature verification at deploy time

For AxisPay's JDK 17 services, this includes Java dependencies, transitive jars, the runtime base image, and the registry artefacts around the image.

2. Why it exists

A Spring Boot service is mostly code you did not write. When a CVE lands, the first task is usually not patching. It is finding where the vulnerable component exists at all. If `fraud-service` pulls in a vulnerable jar transitively, reading `pom.xml` is not enough.

Supply chain controls exist to make exposure visible and provenance defensible.

3. The business problem

AxisPay receives a Log4Shell-style advisory: a vulnerable logging component exists in a transitive dependency of `fraud-service`. Without SBOMs, the platform team starts manual dependency audits across `payment-service`, `fraud-service`, `auth-service`, `merchant-service` and `core-service`. That is hours or days of uncertainty.

With SBOMs tied to image digests, they query the vulnerable package and version, identify every affected service in minutes, and patch only the real exposure set. The difference is not convenience. It is incident speed.

4. How it works

Generate an SBOM during the build or from the final image:

```
<plugin>
  <groupId>org.cyclonedx</groupId>
  <artifactId>cyclonedx-maven-plugin</artifactId>
  <version>2.8.0</version>
</plugin>
```

```
syft registry.example.com/axispay/fraud-service:2026.08.14 -o cyclonedx-json > fraud-service.sbom.json
cosign sign   registry.example.com/axispay/fraud-service@sha256:abcd...
cosign verify registry.example.com/axispay/fraud-service@sha256:abcd...
```

Then enforce at admission: only images signed by the trusted CI identity may run.

Base-image choice belongs in the same discussion:

Runtime image	Benefit	Trade-off
full JDK	easy debugging with shell and JVM tools	larger attack surface
slim JRE	smaller, familiar glibc base	fewer diagnostics
distroless Java	minimal surface, fewer packages to patch	almost no in-container debugging
Alpine Java	small image	musl compatibility surprises for some Java workloads

For AxisPay, distroless or slim glibc-based images are the safe default. Alpine is a deliberate choice, not an automatic upgrade.

5. Internal architecture

Two chains matter:

Chain	Meaning
dependency chain	what components are present
provenance chain	who produced the image

Maven or Gradle resolve the first well. Cosign and the registry record the second by digest.

6. Component interactions

```
git push
   → build JAR
   → generate SBOM
   → build image
   → sign image digest
   → push image, signature, SBOM
   → admission verifies signature
   → pod starts only if policy passes
```

7. Enterprise example

A processor with signing but no SBOMs knew its images were authentic and still needed two days to identify a vulnerable XML library across services. After adding searchable SBOMs, a later advisory took minutes to scope.

8. Real-world analogy

The SBOM is the cargo manifest. The signature is the tamper seal.

Where it breaks

Software cargo is nested. One top-level dependency can pull twenty more. "We do not depend on Log4j directly" proves very little.

9. Best practices

- Generate SBOMs for the build and for the final image.
- Store them against the image digest.
- Sign digests, not tags.
- Enforce verification at admission, not only in CI.
- Prefer distroless or slim glibc-based runtime images for production Java services.

10. Common mistakes

Mistake	What happens
SBOM generated and discarded	no usable inventory during incidents
signing tags	proof disappears when tags move
scanning source only	OS-package CVEs are missed
choosing Alpine only for size	musl compatibility surprises appear later

11. Security considerations

An SBOM does not stop an attack. It shortens the gap between CVE disclosure and exposure knowledge. Image signing addresses a different risk: untrusted or tampered images entering the cluster.

12. Performance considerations

SBOM generation and signing cost CI seconds, not runtime overhead. Signature verification adds small pod-start latency and is operationally cheap compared with running unverified images.

13. High availability

If signature verification depends on a fragile webhook, deployment can stop everywhere. Test policy in non-blocking mode first and keep a documented, audited bypass for registry or signer outages.

14. Disaster recovery

Back up the policy and the inventory:

Artefact	Why
SBOM store	rapid CVE scoping after recovery
signing policy	enforces trusted-image deployment
verification material	required to validate images post-restore

15. Monitoring

Watch for signature-policy rejections and stale base-image digests. An old `eclipse-temurin` or distroless digest is a patch-lag signal even before the next advisory lands.

16. Troubleshooting

Symptom	Cause	Command
pod rejected as unsigned	image not signed or wrong identity	`cosign verify <image>@<digest>`
vulnerable jar missing from SBOM	source-only generation	`mvn dependency:tree`
distroless image hard to inspect	no shell tools	use an ephemeral debug container

Interview questions

1. **Why is an SBOM better than reading `pom.xml` during a CVE incident?** — It includes resolved transitive dependencies.
2. **Why sign digests rather than tags?** — Tags move; digests do not.
3. **What does admission-time verification add beyond CI?** — It blocks untrusted images at deployment time.
4. **Why is Alpine a cautious default for Java?** — Small image, but musl compatibility and support surprises.
5. **What did AxisPay gain during the Log4Shell-style incident?** — Rapid identification of every affected service.

5.9 Container Image Scanning and Vulnerability Management

1. What it is

Image scanning checks the final container for known vulnerabilities in both OS packages and application dependencies.

Layer	Example
OS packages	openssl, glibc, ca-certificates
Java dependencies	Jackson, Netty, Logback, Spring jars

For Java, both layers matter.

2. Why it exists

A rebuilt image can become vulnerable even if application code did not change, because the base image aged. The reverse is also true: a fresh base image does not fix an old transitive jar. Scanning exists to catch both.

3. The business problem

AxisPay's `payment-service` release passes tests, then Trivy reports a critical CVE in Jackson. The team does not depend on Jackson directly. The scanner is still right: the vulnerable jar arrived transitively.

The organisation therefore needs a path from finding to triage to remediation to proof.

4. How it works

```
trivy image --severity HIGH,CRITICAL registry.example.com/axispay/payment-
service:2026.08.14
grype registry.example.com/axispay/payment-service:2026.08.14
```

Useful CI policy:

Severity	Default action
CRITICAL	fail the build
HIGH	fail unless a tracked exception exists
MEDIUM and below	record and patch on schedule

The exception needs an owner and an expiry date.

Triage questions decide urgency:

Question	Why it matters
Is the package actually in the shipped image?	source and runtime do not always match
Is the vulnerable code path reachable?	an exposed parser bug outranks an unused optional module
Is there a fixed version now?	some findings need temporary exception handling

5. Internal architecture

Scanners match discovered packages to vulnerability databases. For Java images they inspect the image layers, jar metadata and OS package database. That is why the scan belongs on the **final image**.

6. Component interactions

```
mvn package
   → build image
   → Trivy / Grype scan image
   → policy gate
   → publish only if policy allows
```

7. Enterprise example

A firm blocked every HIGH finding. One upstream OpenSSL HIGH with no immediate vendor fix stopped every release, and the gate was disabled. The corrected rule was stricter in practice: CRITICAL always blocks; HIGH blocks unless an approved, expiring exception exists.

8. Real-world analogy

A scanner is the airport X-ray machine. It highlights suspicious items. The triage process decides which ones really stop the journey.

Where it breaks

Scanners infer risk from package identity. One finding may be urgent RCE; another may be an unused optional module.

9. Best practices

- Scan the final image, not only the repository.
- Update scanner databases in CI.
- Fail on CRITICAL by default.
- Allow HIGH only with tracked exceptions.
- Re-scan after remediation.

10. Common mistakes

Mistake	What happens
scanning source only	OS CVEs missed
scanning base image only	vulnerable jars missed
permanent exceptions	risk acceptance becomes invisible
fixing `pom.xml` and not rebuilding	vulnerable image still ships

11. Security considerations

A green scan means "no known match now", not "safe forever". A red scan also needs triage: is the package present, on the runtime path, and actually reachable?

12. Performance considerations

Scanning costs CI minutes, not cluster resources. Smaller runtime images scan faster and usually produce fewer irrelevant findings.

13. High availability

If the scanner service is down, releases may stop. Decide that consciously and keep a break-glass, audited bypass for hotfixes.

14. Disaster recovery

Retain historical scan reports, policy definitions and exception records. After an incident, audit often asks when a vulnerable image first became known.

15. Monitoring

Track open CRITICAL findings by service, exception records nearing expiry, and image age since last rebuild. Those three numbers drive most patching work.

16. Troubleshooting

Symptom	Cause	Command
Jackson CVE but no direct dependency	transitive jar	`mvn dependency:tree | grep jackson`
CVE still present after version bump	old image still scanned	rebuild and re-scan
same base CVEs every week	stale parent image	rebuild from newer base

Interview questions

1. **Why scan the final Java image?** — It contains both OS packages and resolved runtime jars.
2. **Why is "fail every HIGH" brittle?** — Some HIGHs have low exploitability or no immediate fix, so teams disable the gate.
3. **What is the first triage question after a finding?** — Is the vulnerable component actually present and reachable?
4. **How would AxisPay fix a transitive Jackson CVE in `payment-service`?** — Override the version in Maven, rebuild, and re-scan.
5. **Why do findings reappear without code changes?** — New advisories land against existing base-image packages.

Worked example — remediating a Jackson CVE in `payment-service`

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.2</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Then prove the fix:

```
mvn -q dependency:tree | grep jackson-databind
trivy image --severity HIGH,CRITICAL registry.example.com/axispay/payment-
service:2026.08.15
```

The evidence is the clean re-scan, not the merged PR.

One subtle benefit of this workflow is cultural: developers learn that a vulnerability report is not an argument with security but a reproducible engineering task. Find the component, decide whether it is reachable, patch it or document the exception, rebuild, and prove the result with a second scan. That repeatable loop is what makes vulnerability management sustainable.

5.10 Audit Logging and PCI-DSS Compliance

1. What it is

Audit logging is the durable record of administrative and business-significant action.

Layer	Records
Kubernetes audit logs	who accessed cluster objects, Secrets, `pods/exec`, RBAC
application audit logs	who changed payment or ledger state

2. Why it exists

PCI-DSS requirement 10 requires traceable access and activity records. Debug logs and cluster events are not enough. An audit log is durable, reviewable and harder to tamper with.

3. The business problem

AxisPay's acquiring partner asks: who read or modified Secrets, who changed RBAC, and who opened a shell in a pod that handles cardholder data last month? Without audit logs, the answer is reconstruction by memory. With them, it is a query.

4. How it works

Kubernetes audit policies support four levels:

Level	Records
`None`	nothing
`Metadata`	who, when, resource, verb, source IP, code
`Request`	metadata plus request body
`RequestResponse`	request and response bodies

For a PCI-scoped payments platform, the useful pattern is selective depth:

Resource / action	Level
`pods/exec`, `pods/portforward`, secret reads	`Metadata`
Secret writes, RBAC changes	`Request`

Application audit records in `payment-service` and `core-service` should capture actor, transaction ID, old state, new state and correlation ID, retained for one year with three months immediately available.

A useful application-audit schema is:

Field	Example
actor	`merchant-api-key:mk_7741`
transaction_id	`pay_7F1K`
old_state	`AUTHORIZED`
new_state	`CAPTURED`
correlation_id	shared cross-service trace
previous_hash / current_hash	tamper-evident chain

5. Internal architecture

API-server audit logging usually flows:

```
API server → audit file or webhook → collector → SIEM / durable store
```

Application audit logging should be append-only and separate from debug logging.

6. Component interactions

```
kubectl exec / get secret / patch rolebinding
   → API server
   → audit event
   → collector
   → durable store

payment or ledger state change
   → application audit record
   → append-only store
```

7. Enterprise example

A processor kept API audit files only on control-plane nodes and rotated them daily. During an investigation, the `exec` evidence had already expired. They had logging, but not retention.

8. Real-world analogy

The audit log is the vault access register. The ledger is the record of what happened to the money.

Where it breaks

Kubernetes audit logs show that `exec` happened. They usually do **not** show every command typed in an interactive shell.

9. Best practices

- Log secret access, `pods/exec`, and RBAC changes explicitly.
- Ship API audit logs off-node immediately.
- Keep application audit logs append-only and tamper-evident.
- Restrict access to audit data.
- Retain one year, with three months hot.

10. Common mistakes

Mistake	What happens
everything at `RequestResponse`	large volume, sensitive over-capture
audit logs only on-node	evidence disappears with rotation
debug logs treated as audit	wrong retention and mutability
assuming `exec` logs shell commands	investigation gap remains

11. Security considerations

Audit logs themselves are sensitive. They expose identities, IPs, object names and sometimes payloads.

The important `kubectl exec` limitation is real:

- API audit logs record the request
- non-interactive commands may appear in the URI
- interactive shell keystrokes usually do not

If AxisPay needs shell-session content, it needs session recording in a bastion or access proxy.

12. Performance considerations

Audit logging adds write volume to the API server. Selective policy matters. Application audit writes should be durable and asynchronous, but not lossy.

13. High availability

The audit pipeline needs replicated collectors and a durable sink. A perfect audit policy with a dead shipping path is a quiet failure.

14. Disaster recovery

Audit data is evidence, not just observability. Recovery must preserve recent searchable records, long-term archive, and the procedure to restore both.

15. Monitoring

Alert on rare actions:

- any `pods/exec` in PCI-scoped namespaces
- any human read of Secrets
- any ClusterRoleBinding change
- audit shipping lag

16. Troubleshooting

Symptom	Cause	Command
secret read missing from audit trail	policy too narrow	inspect audit policy
`exec` event present, commands absent	expected limitation	use session recording
RBAC body missing	level too low	raise RBAC resources to `Request`

Interview questions

1. **What is the difference between `Metadata` and `Request`?** — `Request` includes the request body.
2. **Why audit `pods/exec` in a PCI namespace?** — It is a privileged access path into sensitive workloads.
3. **What is the key limitation of API audit logs for `kubectl exec`?** — Interactive shell commands are usually not captured.
4. **Why are app audit logs still required?** — Kubernetes logs cluster action, not payment-state transitions.
5. **What retention pattern is common for PCI?** — One year retained, three months immediately available.

Worked example — investigating a suspicious `kubectl exec`

An alert shows `pods/exec` against a `payment-service` pod. The investigation asks:

1. who initiated it — username, groups, source IP, user agent
2. what target — namespace, pod, container, response code
3. what possible exposure — mounted Secrets, service function, cardholder-data path
4. what secondary evidence exists — bastion logs or session recording

The useful discipline is knowing what the audit log proves, and what other controls must fill the gap.

5.11 AxisPay RBAC Role Design

1. What it is

This is the concrete RBAC catalogue for AxisPay personas.

2. Why it exists

Least privilege is not a slogan. It is a specific list of verbs, resources and namespaces for named groups.

3. The business problem

An access review found that fraud analysts could read Secrets cluster-wide. The intended requirement had been simple: read `fraud-service` logs in one namespace. The actual grant came from an overly broad ClusterRoleBinding.

4. How it works

Start with personas:

Persona	Namespace	Logs	Exec	Secrets
fraud-analyst	`axispay-risk`	yes	no	no
payments-oncall	`axispay-core`	yes	yes	no
platform-admin	cluster-wide	yes	yes	broad

Example roles:

```
kind: Role
metadata:
  name: fraud-analyst
  namespace: axispay-risk
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log", "events"]
    verbs: ["get", "list", "watch"]
```

```
kind: Role
metadata:
  name: payments-oncall
  namespace: axispay-core
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log", "pods/exec", "events", "services"]
    verbs: ["get", "list", "watch", "create"]
```

```
kind: ClusterRole
metadata:
  name: platform-admin
rules:
  - apiGroups: ["*", ""]
    resources: ["*"]
    verbs: ["*"]
```

And the bindings are intentionally different:

Role	Binding type	Reason
fraud-analyst	RoleBinding	scope stays inside `axispay-risk`
payments-oncall	RoleBinding	incident access only in `axispay-core`
platform-admin	ClusterRoleBinding	genuinely cluster-scoped operations

5. Internal architecture

The useful pattern is define permissions once, bind them narrowly:

Object	Answers
Role / ClusterRole	what may be done
RoleBinding / ClusterRoleBinding	where and by whom

6. Component interactions

```
SSO group claim
   → binding selected
   → rules unioned
   → request allowed or denied
```

The proof is `kubectl auth can-i`, including denials.

7. Enterprise example

A support team reused a broad convenience ClusterRole for every engineer. It solved tickets quickly and quietly granted secret-reading paths everywhere. The eventual fix looked like AxisPay's model: narrow read role, audited incident role, small admin group.

8. Real-world analogy

Fraud analysts read the chart. Payments on-call enters the treatment room. Platform admin holds the building master key.

Where it breaks

Real buildings can deny a person at one door and allow them at another. Kubernetes RBAC has no deny. Any binding that grants a permission wins.

9. Best practices

- Bind groups, not individuals.
- Keep fraud analysis read-only.
- Grant `pods/exec` only to incident or admin roles.
- Never bundle Secret access into log-reading roles.
- Review the aggregate permissions quarterly.

10. Common mistakes

Mistake	What happens
ClusterRoleBinding used by reflex	access leaks to every namespace
`pods/exec` added for convenience	log readers become secret readers in practice
temporary incident grant left behind	emergency access becomes permanent

11. Security considerations

`payments-oncall` is intentionally narrow in namespace and broad in incident power. It has no `secrets` grant and still implies secret access through `pods/exec`, which is why every use must be audited.

12. Performance considerations

The real performance issue is human. A role too narrow makes every incident wait for admin. A role too broad removes the control entirely.

13. High availability

Keep a break-glass admin credential outside the normal IdP path, and avoid making routine support depend on `platform-admin`.

14. Disaster recovery

Store the role catalogue and the SSO group mapping. Restoring YAML without restoring group membership restores objects, not access.

15. Monitoring

The useful review matrix is:

Check	Expected
fraud-analyst can get `pods/log` in `axispay-risk`	yes
fraud-analyst can get `secrets` anywhere	no
payments-oncall can create `pods/exec` in `axispay-core`	yes
payments-oncall can get `secrets` in `axispay-core`	no

16. Troubleshooting

Symptom	Cause	Command
analyst cannot read logs	missing `pods/log` subresource	`kubectl auth can-i get pods/log ...`
on-call cannot exec	wrong verb on `pods/exec`	grant `create`
user has access outside intended namespace	accidental ClusterRoleBinding	inspect bindings

Interview questions

1. **Why should `fraud-analyst` not include `pods/exec`?** — Because exec is a path to secret values and admin power.
2. **Why can `payments-oncall` exec but not read Secrets directly?** — Incident response needs shell access in one namespace; direct secret browsing remains disallowed.
3. **What caused the AxisPay least-privilege failure?** — An overly broad ClusterRoleBinding.
4. **Why bind groups rather than users?** — Easier review, removal and rotation.
5. **What is the proof after fixing an over-grant?** — `kubectl auth can-i` for the intended allows and denials.

Worked example — fixing the least-privilege violation

The bad state was a ClusterRoleBinding granting fraud analysts `get secrets` cluster-wide. The fix is:

1. remove the broad ClusterRoleBinding
2. create the `fraud-analyst` Role in `axispay-risk`
3. bind it only in that namespace
4. prove:

```
kubectl auth can-i get pods/log -n axispay-risk --as=fraud.analyst@axis.example
kubectl auth can-i get secrets  -n axispay-risk --as=fraud.analyst@axis.example
kubectl auth can-i get secrets  -n axispay-core --as=fraud.analyst@axis.example
```

Expected: `yes`, `no`, `no`.

5.12 Incident Response Playbook for AxisPay

This section is a runbook, not a theory chapter. The common rules are simple:

Principle	Why
stabilise first	the system must stop getting worse
preserve evidence	payments incidents become compliance questions quickly
define scope early	leadership wants counts, time window and exposure

Scenario A — unauthorised use of an `auth-service` API token

The signal is anomalous traffic: a token that normally reads merchant profile data now drives bursts of session and validation requests from a new network.

The response path is:

1. open the incident and assign commander, comms owner and technical lead
2. identify the token, time window, source IPs, endpoints and affected clients
3. revoke or rotate the token immediately
4. query `auth-service` and downstream audit logs to see what it touched
5. check Kubernetes audit logs for secret reads, RBAC changes, `pods/exec` or image changes around the same time
6. notify compliance if cardholder data may have been touched
7. remove the leak path — code, log sink, Secret exposure or credential handling flaw
8. restore service only after replacement credentials are distributed and tested

The mistake to avoid is treating revocation as closure. It stops active misuse. It does not answer what data was reached or how the token leaked.

Questions leadership will ask quickly:

Question	Evidence
which clients were affected?	`auth-service` access and audit logs
was cardholder data touched?	downstream `payment-service` and `core-service` trails
how long was exposure open?	first anomalous event to revocation time

Scenario B — payment integrity violation: double charge or ledger mismatch

The signals are duplicate captures, merchant complaints, or reconciliation drift between `payment-service` and `core-service`.

The response path is:

1. treat it as Sev-1 until proven otherwise

2. freeze the faulty path — fail readiness, disable a consumer, or scale the deployment to zero if necessary
3. capture version, config, feature-flag state and queue offsets before rollback
4. define affected merchants, payment IDs, currencies and time window
5. reconcile the truth set: gateway requests, payment events, acquirer responses and `core-service` ledger entries
6. stop the faulty automation, often a retry path or idempotency defect
7. execute compensating transactions: reversals, refunds or corrective ledger entries
8. declare recovery only when the money and ledger agree
9. add detection for recurrence: duplicate-charge ratio, idempotency-key reuse, reconciliation lag

The reconciliation set is always larger than one service:

- payment API request log
- idempotency key history
- acquirer response log
- `payment-service` event trail
- `core-service` ledger entries
- merchant-visible order status

What finance, engineering and support each need is different:

Team	Immediate question
engineering	what code path or component is wrong right now?
finance-ops	which transactions require reversal, refund or manual review?
merchant support	which merchants need proactive communication before they discover it themselves?

Containment choices:

Option	Use when
fail readiness	traffic must stop, pods should remain for forensics
scale to zero	code is actively harmful
disable consumer or feature flag	background path is faulty, reads can continue

Communications and evidence discipline

Both scenarios need:

- a single incident channel
- UTC timeline
- affected merchants or transactions
- audit-log extracts

- exact remediation times

If cardholder data or financial integrity is in scope, compliance joins early, not at the end.

Evidence preservation is a task in its own right:

Evidence	Why keep it early
rollout history	connects the incident to a version or config change
feature-flag state	many integrity incidents are controlled by flags rather than code deploys
audit-log slice	proves administrative actions around the incident window
queue offsets or batch IDs	required to replay or reconcile safely
merchant examples	turns a vague symptom into reproducible evidence

The incident is not over when the pager quiets down. AxisPay closes only when three conditions are true:

Closure condition	Token misuse case	integrity case
active harm stopped	token revoked or rotated	faulty processing path frozen
scope known	affected clients and data paths identified	affected transactions and ledger delta quantified
durable fix queued	credential-handling control change agreed	code, reconciliation or alerting change agreed

The post-incident review should produce five concrete outputs:

1. exact root cause
2. exposure window
3. merchant or transaction count
4. control failure and control improvement
5. alert or runbook change that would shorten the next incident

Post-incident outputs that matter

The review must produce control changes:

Incident	Control improvement
stolen token	shorter token lifetime, geography anomaly alert, rotation runbook
double charge / ledger mismatch	stronger idempotency, automated reconciliation checks, compensating-transaction playbook

An incident without a control change is usually rehearsal for the next one.

Day 5 cheat sheet

Identity and admission

```
# who is this pod, and does it carry a token?
kubectl get pods -A -o custom-columns='POD:.metadata.name,SA:.spec.serviceAccountName'
kubectl exec -n axispay-core <pod> -- ls /var/run/secrets/kubernetes.io/serviceaccount/

# Pod Security
kubectl get ns -L pod-security.kubernetes.io/enforce
kubectl label ns axispay-core pod-security.kubernetes.io/enforce=restricted --overwrite
```

RBAC

```
kubectl auth can-i list pods   -n axispay-core --as=auditor@axis.example
kubectl auth can-i get secrets -n axispay-core --as=auditor@axis.example
kubectl auth can-i --list      -n axispay-core --as=auditor@axis.example
kubectl auth can-i list nodes  --as=system:serviceaccount:axispay-ops:node-agent

# every ClusterRole that can read Secrets
kubectl get clusterrole -o json | jq -r '.items[] | select(.rules[]? |
  select((.resources[]? == "secrets" or .resources[]? == "*") and
         (.verbs[]? == "get" or .verbs[]? == "list" or .verbs[]? == "*")))
  | .metadata.name' | sort -u
```

Helm

```
helm template axispay ./charts/axispay | grep '^kind:' | sort | uniq -c
helm lint ./charts/axispay -f charts/axispay/values-prod.yaml
helm upgrade --install axispay ./charts/axispay --atomic --timeout 10m
helm history axispay
helm rollback axispay 3 --wait
helm diff upgrade axispay ./charts/axispay -f charts/axispay/values.yaml

# what a release actually is
kubectl get secret -n default -l owner=helm
```

PromQL

```
# traffic
sum by (service) (rate(axispay_http_requests_total[5m]))

# error ratio, 5xx only
sum(rate(axispay_http_requests_total{status=~"5.."}[5m]))
  / sum(rate(axispay_http_requests_total[5m]))

# p99 latency
histogram_quantile(0.99, sum by (le) (
  rate(axispay_http_request_duration_seconds_bucket{service="payment-service"}[5m])))

# business outcome
sum by (status) (rate(axispay_payments_total[5m]))

# the alert that fires on silence
sum(rate(axispay_payments_total[10m])) == 0
```

LogQL

```
{namespace="axispay-core"}                                    # start here, always
{namespace=~"axispay-.*"} | json | level="error"
{namespace=~"axispay-.*"} | json | correlation_id="<id>"
{namespace=~"axispay-.*"} | json | duration_ms > 500
sum by (service) (rate({namespace="axispay-core"} |= "error" [5m]))
```

Alert routing

```
kubectl get alertmanagerconfig -n axispay-observability
kubectl -n axispay-observability port-forward svc/alert-sink 8080:8080
curl -s localhost:8080/api/v1/routes | jq .
curl -s 'localhost:8080/api/v1/alerts?channel=payments' | jq -r '.alerts[].alertname'
```

Cluster upgrades

```
kubeadm upgrade plan                    # what is available, and what it will do
kubeadm upgrade apply v1.36.2           # control plane FIRST, always
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>

# what will break at the next upgrade — check this months ahead
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis

# the skew, live
kubectl get nodes -o custom-
columns='NODE:.metadata.name,KUBELET:.status.nodeInfo.kubeletVersion'
```

Offline validation — no cluster required

```
python3 platform/admin/validate/check-helm-chart.py     # 94 chart assertions
python3 platform/admin/validate/check-promql.py         # 47 expressions parsed
python3 platform/admin/validate/simulate-rbac.py        # 28 RBAC assertions
python3 platform/admin/validate/simulate-netpol.py      # 46 policy assertions
```

Day 5 review questions

1. A pod uses the `default` ServiceAccount and RBAC grants it nothing. Why is the mounted token still a problem?
2. What is the difference between a 401 and a 403 when using a pod's token against the API server?
3. You set `enforce: restricted` on a namespace with running non-compliant pods. What happens, and when do you find out?
4. Why can a log collector not run in a namespace enforcing `baseline`?
5. How do you revoke a permission in Kubernetes RBAC?
6. Name two ways to read a Secret with no `secrets` grant at all.
7. Why is `ClusterRole` + `RoleBinding` the most useful of the four combinations?
8. What is stored in a Helm release Secret, and what follows from that for rollback?
9. Why must `.spec.selector` contain only stable labels, and when does violating this fail?
10. Why does the AxisPay chart omit `replicas` for `payment-service`?
11. Name one setting that must be identical in dev, staging and production, and say why.
12. Why is the average latency the wrong number for an SLO?
13. A Prometheus target is missing rather than down. What does that tell you?
14. Why does a ServiceMonitor select a Service rather than pods?
15. What happens if you make `correlation_id` a Loki label?
16. What does Alertmanager do that Prometheus deliberately does not?
17. Write the query that detects "everything is green and no payments are arriving".
18. Given only a merchant's payment reference, how do you produce the cross-service log trail?
19. Why must the control plane be upgraded before the kubelets?
20. A drain has been running for twenty minutes. What is the correct next step, and what is the wrong one?

Answers: `documents/assessments/answer-keys/day5-answer-key.md`

Day 5 summary

You began the week able to deploy an application. You end it able to **operate** one.

The platform now has an identity model where every workload is named and only one carries a credential; an authorisation model where every grant and every denial can be printed for an auditor; a packaging model where the whole platform installs in one command in any of five configurations; and an observability model where the SLO is a query, a latency spike leads to the log line that caused it, and nine alerts fire on symptoms a merchant would actually feel.

The through-line worth carrying out of the week is the correlation ID. You implemented it on Monday, in `edge-gateway`, for no visible reason. On Friday it is what turns "seven services touched this payment" into one sorted list with the slow one obvious.

Most of the decisions that make a system operable look unnecessary at the moment you make them. Recognising which ones are worth making anyway is a large part of what seniority means.